

FINDING

§11.4.124 dead-code investigation + §11.4.6 classification — FINDING

Revision: 1 **Last modified:** 2026-06-16T16:00:12Z **Mode:** READ-ONLY, NON-COMMITTING investigation (separate committer agent is sole committer per §11.4.84). No edits/stages/commits made by this agent. **HEAD at investigation:** c2aa7c8 **Discipline:** superpowers:systematic-debugging Phase-1 forensic + §11.4.124 investigate-before-remove + §11.4.6 no-guessing.

**TASK 1 — pkg/api/server.go Server type —
VERDICT: NEVER-WIRED scaffolding (do NOT
delete on sight; recommend leave + document, or
tracked-Obsolete)**

Git-history FACT chain

- `git log --follow pkg/api/server.go` → only 3 commits touch it: 76b94dc (Auto-commit — **first appearance**, `--diff-filter=A` confirms it is the add commit), e57fe22 (Comprehensive test/lint fixes), 85d9849 (§11.4.115 fix: `Server.Stop()` was a no-op placeholder — a real bug-fix landed against this type).
- `git grep -l 'api.NewServer(' <commit> -- 'cmd/*'` for 76b94dc, 96e780e, e57fe22, 7a59eff → **EMPTY in every commit**. `api.NewServer()` was **never present in any cmd/ entry across the entire history**.
- The first-appearance commit 76b94dc added `pkg/api/server.go` (214 lines) **alongside *_comprehensive_test.go files only** (`cmd/cli`, `cmd/deployment`, `cmd/markdown-translator`, `cmd/server`, `cmd/translate-ssh` test scaffolds). It was born as test-targeted scaffolding, never with a production call-site.

"No references" — real or hidden? (FACT)

- Non-test callers of `api.NewServer` / `api.Server` / `SetTranslator` in `cmd/` + `pkg/`: **ZERO**. (The two `NewServer` `grep` hits are unrelated: `translatorgrpc.NewServer` in `pkg/grpc` and `stdlib grpc.NewServer` — different package, different type.)

- Literal `api.NewServer()` hits in the tree are NOT our type:
 - `llms_verifier/llm-verifier/...` → the LLMsVerifier **submodule's own** `api` package, 2-arg signature `api.NewServer(cfg, db)` — NOT `digital.vasic.translator/pkg/api.NewServer(config ServerConfig)`.
 - `test/security/input_validation_new_test.go` → **test-only** caller of OUR `api.NewServer(api.ServerConfig{...})`.
- No hidden wiring: no reflection / build-tags / DI / plugin-registry / codegen reaches `api.Server`. The two production HTTP servers are wired through **entirely different** types (see below). UNCONFIRMED on nothing material here — the static + history evidence is conclusive that no production path constructs `api.Server`.

What actually serves HTTP in production (FACT — proves `api.Server` is superseded scaffolding)

- `cmd/server/main.go` → `api.NewHandler(cfg, eventBus, cache, authService, wsHub, distributedManager)` (`pkg/api/handler.go`, the rich `Handler`) + `api.InitVerifierFromConfig` + its own `http.Server` + HTTP/3. **Does not touch `api.Server`.**
- `cmd/api-server/main.go` → `APIServer` struct with its own `*http.Server`, `gRPC` client, `api.NewVerifierHandler`. **Does not touch `api.Server`.**
- `api.Server` is a parallel, simpler gin server (`/health`, `/api/translate`, `/api/languages`, `/api/stats`, `/api/upload`, `/api/batch`) with placeholder handlers (`statsHandler` returns `translations:0`; `languagesHandler` static). It is functionally **superseded** by `Handler` (richer `/api/v1/*`) — but was never a wired predecessor that got orphaned; it was scaffolded in parallel and never wired.

Classification (§11.4.6 FACT, not guess)

NEVER-WIRED. It was born as test-targeted scaffolding (first commit added it with comprehensive test files, never a `cmd/` call-site), it has only test callers today, and the two production servers were always built on different types (`Handler` / `APIServer`). It is NOT ORPHANED (no evidence a production call-site ever existed and was removed) and NOT a from-wired-to-dead regression.

Recommendation (§11.4.124 — NOT blind deletion)

1. **Do NOT blind-delete.** It carries a §11.4.115 bug-fix commit (85d9849 `Stop()`) + a live test suite (`server_test.go`, `server_lifecycle_test.go`, `server_realhttp_integration_test.go`, `test/security/input_validation_new_test.go`) that genuinely exercise it — deletion would also remove that real test coverage and the security input-validation tests that depend on it.
2. **Preferred: LEAVE + DOCUMENT** as an intentionally-test-only HTTP surface (a lightweight in-process gin server used by the security/integration test suites), OR — if the project wants it gone — route it through the §11.4.90 tracked **Obsolete** (→ **Fixed.md**) path with reason superseded-by-design-change (superseded by `pkg/api.Handler`), its **own separate descriptive commit** citing this git-history evidence (§11.4.124), AND migrate the dependent security tests onto `Handler` first so coverage is not lost.

3. **§11.4.122:** `api.Server` is not an end-user-facing shipped capability (no `cmd/` ships it), so operator keep-or-remove confirmation is advisory, not mandatory — but the `Obsolete` route still requires the §11.4.124 `separate-commit-with-proof` discipline.
 4. **Reject** the two streams' "UNWIRED dead-code $\Rightarrow$ delete" framing: correct on "unwired", but deletion-on-sight is forbidden here because (a) test coverage depends on it, (b) the correct closure is tracked-Obsolete-with-test-migration, not silent removal.
-

TASK 2 — GET `/api/v1/providers` static list — VERDICT: REAL BUG (mis-represents capability AND availability)

FACT — there are TWO distinct `/providers` endpoints

1. `cmd/api-server/main.go:540 getProviders` $\rightarrow$ proxies the **real gRPC** `GetProviders` RPC $\rightarrow$ `pkg/grpc/server.go:399` returns `s.providers.GetAll()` — a genuine dynamic registry source. **Not the finding.**
2. `pkg/api/handler.go:638 listProviders` (the `/api/v1/providers` route registered at `handler.go:144`, served by `cmd/server`) $\rightarrow$ returns a **hardcoded static list** of exactly 4 providers: `openai` (`gpt-4`, `gpt-3.5-turbo`), `anthropic` (`claude-3-sonnet-20240229`, `claude-3-opus-20240229`), `zhipu` (`glm-4`), `deepseek` (`deepseek-chat`). **This is the wave9 finding.**

Why it is a real bug (FACT)

- **Mis-represents capability.** The real provider factory `NewLLMTranslator` (`pkg/translator/llm/llm.go:264+`) supports **20+** providers: `openai`, `anthropic`, `zhipu`, `deepseek`, **`qwen`, `gemini`, `groq`, `cohere`, `mistral`, `xai`, `replicate`, `cerebras`, `cloudflare`, `siliconflow`, `hyperbolic`, `togetherai`, `sambanova`, `kim`, `novita`, `nlpcloud`, ...** The static list omits 16+ supported providers and ships stale model ids (`claude-3-opus-20240229` etc.).
- **Mis-represents availability.** The `Handler` already holds `config.Config` (`handler.go:41`), whose `Translation.Providers` `map[string]ProviderConfig` and `Verification.Providers []string` (`internal/config/config.go:50,95,159`) are the real configured/enabled source — but `listProviders` ignores them and returns the same 4 regardless of what is actually configured. A client calling `/api/v1/providers` is told `openai` is available even with no key configured, and is NOT told `gemini/qwen` (configured + supported) exist. A client cannot drive a provider-selection UI from this.
- **A dynamic real source exists** and is the contrast oracle: `GET /api/v1/verified-models` + `GET /api/v1/providers/verified` (`pkg/api/verifier_handlers.go:100,104,248`) derive providers from `client.GetVerifiedModels(ctx)` (`LLMsVerifier` — the project's SSoT per `CLAUDE.md`). The static `listProviders` is the only providers endpoint on `cmd/server` that lies.
- §11.4.6: this is FACT (hardcoded slice literal `handler.go:639-664`, no config read), not a guess. UNCONFIRMED: whether any current dashboard/client

actually consumes `/api/v1/providers` for availability decisions vs the verified endpoints — needs a consumer grep at fix time, but the endpoint's contract ("list available providers" per its doc comment) is violated regardless of current consumers.

Bug-or-intentional: BUG (not an acceptable capability catalogue)

Its own doc comment says "lists **available** translation providers" — an availability claim, not a static catalogue claim — and it ships a partial, stale, config-blind list. Even read charitably as a catalogue, it is wrong (16+ providers missing). Not intentional/acceptable.

Fix recommendation (draft)

Serve real configured/available providers instead of a hardcoded slice:

- Derive the provider set from `h.config.Translation.Providers` (configured) and/or the verified-models source (`verifier_handlers` already does this via `GetVerifiedModels`), reporting per-provider configured/available/requires_api_key from actual config + key presence — not literals.
- Reconcile model lists against the real factory/verifier, not frozen string literals.
- Consider folding `/api/v1/providers` to delegate to the same source as `/api/v1/providers/verified`, or clearly document it as a static capability catalogue AND complete it to the full `llm.go` provider set (the lesser fix).

RED-test shape (§11.4.115 reproduce-first, for the committer/fix agent)

RED_MODE=1 on current artifact MUST FAIL, proving the lie:

- Configure config with providers {gemini, qwen} and NO openai key → GET `/api/v1/providers` → assert response **does NOT** contain gemini/qwen and **does** contain openai (current static behaviour) → that assertion-of-the-bug passes on the broken artifact (RED present).
 - Flip RED_MODE=0 after fix: assert response reflects the configured set (gemini,qwen present; unconfigured openai absent or flagged configured:false). Register as a permanent §11.4.135 regression guard.
-

Summary

- **TASK 1:** `api.Server` is **NEVER-WIRED** scaffolding (git-history FACT: no cmd/ call-site ever; only test callers; superseded by `Handler/APIServer`). Recommend **leave + document OR tracked-Obsolete with test migration in its own §11.4.124 commit** — NOT blind deletion (real test coverage depends on it).
- **TASK 2:** `/api/v1/providers` static list (`handler.go:listProviders`) is a **REAL BUG** — mis-represents both capability (4 of 20+ providers, stale models) and availability (ignores `config.Translation.Providers` it already holds). Fix: serve real configured/available providers; RED-test shape provided.

No bluff. FACTs sourced from git history + code reads at HEAD c2aa7c8.
UNCONFIRMED items marked.